



Università  
di Catania

DEPARTMENT OF ECONOMICS AND BUSINESS

DEPARTMENT OF ELECTRICAL ELECTRONIC AND COMPUTER  
ENGINEERING

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

MASTER'S DEGREE IN DATA SCIENCE

---

# Keyboard Audio Detection

Deep Learning Final Project

*Author:*

Mauro VENTICINQUE

Matricola Number:

1000025992

*Professor:*

Sebastiano BATTIATO

---

ACADEMIC YEAR 2025/2026



Mauro VENTICINQUE

# KEYBOARD AUDIO DETECTION

*Final Project*

UNIVERSITY OF CATANIA

October 2025-January 2026



UNIVERSITÀ DEGLI STUDI DI CATANIA

Department of Economics and Business , Department of Electrical  
Electronic and Computer Engineering , and Department of  
Mathematics and Computer Science

Data Science

## **Keyboard Audio Detection**

by Mauro VENTICINQUE

# ABSTRACT

Acoustic Side-Channel Attacks (ASCAs) represent an emerging class of security threat in which an external adversary attempts to reconstruct typed input by analysing the acoustic emissions of a keyboard. This report investigates the feasibility of such an attack through the lens of deep learning, presenting and comparing three neural architectures: a Temporal Convolutional Network (TCN), a Bidirectional Long Short-Term Memory network (LSTM), and a Convolutional Recurrent Neural Network (CRNN). All models are trained on a dataset of mechanical keyboard audio samples, supplemented by a data augmentation pipeline to compensate for the limited dataset size. The TCN achieves the highest test accuracy of 89%, with a Macro-F1 score of 0.887, outperforming the LSTM (80%, F1: 0.748) and CRNN (83%, F1: 0.742) while maintaining a compact parameter footprint of approximately 399k. These results confirm that acoustic keylogging via deep learning is a realistic practical threat, even under constrained data conditions.



# Contents

|                                                             |            |
|-------------------------------------------------------------|------------|
| <b>Abstract</b>                                             | <b>iii</b> |
| <b>1 Introduction</b>                                       | <b>1</b>   |
| 1.1 Project Goal and Research Question . . . . .            | 2          |
| 1.2 Report Structure . . . . .                              | 3          |
| <b>2 Dataset and Preprocessing</b>                          | <b>5</b>   |
| 2.1 Raw Dataset . . . . .                                   | 5          |
| 2.2 Peak Detection . . . . .                                | 6          |
| 2.3 Audio Splitting . . . . .                               | 7          |
| 2.4 Data Augmentation . . . . .                             | 8          |
| 2.4.1 Additive Gaussian Noise . . . . .                     | 8          |
| 2.4.2 Time Shifting . . . . .                               | 9          |
| 2.4.3 Amplitude Scaling . . . . .                           | 9          |
| 2.5 Feature Extraction: Mel Spectrogram . . . . .           | 10         |
| <b>3 Architectures</b>                                      | <b>13</b>  |
| 3.1 Common Building Blocks . . . . .                        | 13         |
| 3.1.1 Causal Convolutions and Residual Blocks . . . . .     | 13         |
| 3.1.2 Bidirectional Recurrent Blocks . . . . .              | 14         |
| 3.2 Temporal Convolutional Network (TCN) . . . . .          | 15         |
| 3.3 Long Short-Term Memory Network . . . . .                | 15         |
| 3.4 Convolutional Recurrent Neural Network (CRNN) . . . . . | 16         |
| <b>4 Experiments, Results and Conclusion</b>                | <b>19</b>  |
| 4.1 Experimental Setup . . . . .                            | 19         |
| 4.2 Results . . . . .                                       | 20         |
| 4.3 Discussion . . . . .                                    | 21         |
| 4.4 Conclusion and Future Work . . . . .                    | 22         |
| <b>A Code Listings</b>                                      | <b>25</b>  |

|                |           |
|----------------|-----------|
| <b>B Plots</b> | <b>27</b> |
|----------------|-----------|

|                     |           |
|---------------------|-----------|
| <b>Bibliography</b> | <b>31</b> |
|---------------------|-----------|



# Chapter 1

## Introduction

The proliferation of digital devices and the prevalence of keyboard-based input have made keystroke security an increasingly relevant area of concern. Traditionally, the primary threat model for keystroke interception has been software-based key-loggers: a malicious program residing on the victim's operating system and capturing input at the driver level. Such attacks, while effective, require local code execution and are therefore detectable by standard protection tools.

A subtler and harder-to-detect threat vector has emerged in the form of **Acoustic Side-Channel Attacks (ASCAs)**. Rather than compromising the victim's system, an ASCA operates entirely from the outside: an attacker records the acoustic emissions produced by keystrokes and uses signal processing or machine learning to infer the characters typed. The feasibility of this approach was first established by Asonov and Agrawal [1], who demonstrated that keyboard acoustic emanations leak information about which key was pressed. Zhuang et al. [2] later showed

that keystroke classification from audio alone is achievable even without labeled training data, using unsupervised clustering combined with post-processing language models. More recently, Harrison et al. [3] and Spata et al. [4] demonstrated that a deep learning classifier trained on smartphone-recorded keystrokes can exceed 90% character-level classification accuracy, highlighting the practical accessibility of the attack with commodity hardware.

The acoustic signatures of individual keystrokes arise from the mechanical interaction of key, switch, and chassis, and are therefore partially unique to each key’s position on the keyboard. Prior works confirm that short-duration transient sounds carry sufficient discriminative information for fine-grained classification [5]. The signal structure is dominated by a sharp impulsive onset followed by a resonant decay, which is a profile well-suited to time-frequency representations such as the Mel Spectrogram [6].

## 1.1 Project Goal and Research Question

The central research question of this project is: *can a modern deep learning system accurately classify individual keystrokes from their acoustic signatures, using only a small, single-environment dataset?* Three neural architectures (convolutional, recurrent, and hybrid) are designed, trained, and compared to answer this question, alongside a dedicated preprocessing and augmentation pipeline developed to handle the raw audio data.

Answering this question affirmatively would confirm that ASCAs are not merely a laboratory phenomenon, but a realistic everyday privacy risk for billions of users worldwide who type in shared environments

such as offices, cafes, or libraries, locations where a nearby microphone, whether on a smartphone or a voice assistant, is commonplace.

## 1.2 Report Structure

The remainder of this report is organized as follows. Chapter 2 describes the dataset, the peak detection and splitting pipeline, the data augmentation strategy, and the feature extraction process. Chapter 3 presents the three neural architectures and their common building blocks. Chapter 4 covers the experimental setup, quantitative results, and a comparative discussion of the performance of the models. Conclusions and future work are also drawn in Chapter 4.



## Chapter 2

# Dataset and Preprocessing

### 2.1 Raw Dataset

The original dataset [7] consists of audio recordings of alphanumeric keystrokes collected from an **Akko 306B Plus ISO** mechanical keyboard. A total of 36 keys were recorded, each pressed 25 times, yielding **900 raw audio samples**. Recordings were captured in stereo at 48 kHz using a smartphone microphone placed to the left of the keyboard at a distance of 17 cm. The test set consists of 9 complete passwords typed sequentially.

A key limitation of this dataset is its environmental specificity. As noted extensively in the ASCA literature [1], [2], acoustic keystroke signatures are highly sensitive to the recording context: variations in keyboard model, microphone placement, distance, and room acoustics are expected to affect classification accuracy. This work therefore aims to evaluate performances under controlled conditions rather than to generalize to arbitrary real-world scenarios.

## 2.2 Peak Detection

The raw recordings each contain 25 keystrokes separated by silence or low-level ambient noise. To extract individual samples, a peak detection algorithm was used [4] (Algorithm 1), leveraging the Continuous Wavelet Transform (CWT) for transient localization.

The CWT, implemented via `pywt.cwt` [8], decomposes the signal simultaneously across frequency scales and time, making it well-suited for detecting brief impulsive events that span multiple frequency bands. Scales 1 through 127 are used, providing a broad multi-resolution view of the signal. A dynamic threshold is computed as five times the mean absolute coefficient magnitude over the first second of the recording, serving as a baseline estimate of the ambient noise floor. The algorithm then traverses the coefficient matrix column by column: at each time index  $i$ , it checks whether the frame-to-frame difference  $|C_{j,i} - C_{j,i-1}|$  exceeds the threshold for any frequency scale  $j$ . If so, the corresponding timestamp is recorded as a peak. After each detection, the scan advances by  $\lfloor \text{gap\_time} \cdot f_s \rfloor$  samples (`gap_time` = 0.5 s) to skip the acoustic decay of the current keystroke and prevent double triggering.

---

**Algorithm 1:** Peak Detection

---

**Input** : Audio file path  $f$ , wavelet type, output directory, $\text{gap\_time} = 0.5 \text{ s}$ **Output:** Segmented keystroke .wav files

```

1 Load signal  $y$  and sample rate  $f_s$  from  $f$  using librosa [6];
2 Compute CWT coefficients  $C \in \mathbb{R}^{127 \times T}$  via pywt.cwt [8] over
  scales  $1 \dots 127$ ;
3  $\text{noise\_level} \leftarrow \frac{1}{127 \cdot f_s} \sum_{j, t \leq f_s} |C_{j,t}|$ ;
4  $\text{threshold} \leftarrow 5 \times \text{noise\_level}$ ;
5  $\text{peak\_times} \leftarrow []$ ,  $i \leftarrow 0$ ;
6 while  $i < T$  do
7      $\text{detected} \leftarrow \text{false}$ ;
8     for  $j \leftarrow 0$  to 126 do
9         if  $i > 0$  and  $|C_{j,i} - C_{j,i-1}| > \text{threshold}$  then
10             Append librosa.samples_to_time( $i, f_s$ ) to
               $\text{peak\_times}$ ;
11              $i \leftarrow i + \lfloor \text{gap\_time} \cdot f_s \rfloor$ ;
12              $\text{detected} \leftarrow \text{true}$ ;
13             break;
14     if not detected then  $i \leftarrow i + 1$ ;
15 Call SPLITAUDIO( $f, \text{peak\_times}, \text{gap\_time}$ );

```

---

## 2.3 Audio Splitting

Given the list of peak timestamps produced by Algorithm 1, the `SPLITAUDIO` [4] procedure (Algorithm 2) extracts fixed-length segments using `pydub` [9]. Each segment begins exactly at the detected peak time and extends for  $\text{gap\_time} - 0.1 = 0.4$  seconds, capturing the full keystroke

event including its resonant tail, while avoiding overlap with the following keystroke.

---

**Algorithm 2:** SplitAudio

---

**Input** : File path  $f$ , peak times  $\mathcal{P}$ , `gap_time` = 0.5 s, output directory

**Output:** Segmented keystroke .wav files

```

1 Load full audio  $A$  from  $f$  using pydub.AudioSegment [9];
2 for  $(i, t_p) \in \text{enumerate}(\mathcal{P})$  do
3      $t_{\text{start}} \leftarrow \max(0, \lfloor t_p \times 1000 \rfloor)$  ms;
4      $t_{\text{end}} \leftarrow \lfloor (t_p + \text{gap\_time} - 0.1) \times 1000 \rfloor$  ms;
5      $\text{segment} \leftarrow A[t_{\text{start}} : t_{\text{end}}]$ ;
6     Export segment as <name>_i.wav to output directory;
```

---

## 2.4 Data Augmentation

With only 900 original samples, the dataset is insufficient to train a deep neural network without strong regularisation. Data augmentation is a well-established remedy in audio classification [10], [11], and was applied here through three signal-level transformations (Algorithm 3), exclusively on the training split.

### 2.4.1 Additive Gaussian Noise

White noise is synthesised and added to each sample. The noise standard deviation is derived from a target Signal-to-Noise Ratio (SNR) drawn uniformly from `[noise_factor_high, noise_factor_low]` = [5, 15] dB:

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left( \frac{\mathbb{E}[\text{Sig}^2]}{\mathbb{E}[\text{Noise}^2]} \right) \implies \sigma_{\text{noise}} = \sqrt{\frac{\mathbb{E}[\text{Sig}^2]}{10^{\text{SNR}/10}}} \quad (2.1)$$



### 2.4.2 Time Shifting

The waveform is shifted along the time axis by a random integer offset  $\delta$  drawn uniformly from  $[-0.4L, 0.4L]$ , where  $L$  is the number of samples. Positive shifts pad the left with zeros and truncate the right; negative shifts do the reverse. This simulates variability in typing speed and inter-keystroke timing.

### 2.4.3 Amplitude Scaling

The signal is multiplied by a scalar drawn uniformly from  $[0.5, 1.2]$ , simulating differences in typing force and microphone sensitivity. The result is clipped to  $[-1.0, 1.0]$  to prevent numerical overflow. The combination of noise injection and amplitude variation follows the augmentation strategy of Salamon and Bello [11], who demonstrated its effectiveness for environmental sound classification under data-scarce conditions.

**Algorithm 3:** Data Augmentation

**Input** : Input directory, output directory, augmentations per  
file  $N$

**Output:** Augmented .wav files

```

1 for each file  $f \in \text{indir}$  with extension .wav do
2   Load signal  $x$  and sample rate  $f_s$  using librosa;
3   for  $i \leftarrow 1$  to  $N$  do
4      $\triangleright$  Additive Gaussian Noise
5     Sample  $\text{SNR} \sim \mathcal{U}(15, 30)$  dB;
6      $\sigma \leftarrow \sqrt{\mathbb{E}[x^2] / 10^{\text{SNR}/10}}$  (see Eq. 2.1);
7      $\varepsilon \sim \mathcal{N}(0, \sigma^2)^{|x|}$ ,  $\tilde{x} \leftarrow x + \varepsilon$ ;
8      $\triangleright$  Time Shift
9     Sample  $\delta \sim \mathcal{U}_{\mathbb{Z}}(-0.4|x|, 0.4|x|)$ ;
10    if  $\delta > 0$  then  $\tilde{x} \leftarrow \text{pad\_left}(\tilde{x}, \delta)[ : |x| ]$ ;
11    else  $\tilde{x} \leftarrow \text{pad\_right}(\tilde{x}, -\delta)[-|x| : ]$ ;
12     $\triangleright$  Amplitude Scale
13    Sample  $s \sim \mathcal{U}(0.5, 1.2)$ ;
14     $\tilde{x} \leftarrow \text{clip}(s \cdot \tilde{x}, -1.0, 1.0)$ ;
15    Write  $\tilde{x}$  to aug_<name>_i.wav using soundfile;
```

## 2.5 Feature Extraction: Mel Spectrogram

Following augmentation, each raw waveform is processed by the `KeyAudioDataset` class into a 2D Mel Spectrogram. The use of Mel Spectrograms as input features for audio classification is well established [6], [12]: by mapping the linear frequency axis to the perceptually-motivated Mel scale, they compress redundant high-frequency information while preserving discriminative spectral structure. The processing pipeline per sample is:

1. **Mono conversion:** stereo channels are averaged ( $y \leftarrow \frac{y_L + y_R}{2}$ ) to produce a single-channel signal.
2. **Resampling:** to ensure consistency in case of mixed-source data, all audio is explicitly resampled to  $f_s = 48$  kHz using `torchaudio.functional.resample`.
3. **Length normalisation:** for mixed-source consistency, signals longer than  $\lfloor f_s \times 0.5 \rfloor$  samples are truncated; shorter signals are zero-padded to the same target length.
4. **Mel Spectrogram:** computed via `torchaudio.transforms.MelSpectrogram` with  $N_{\text{FFT}} = 2048$  (the `librosa` default [6]), `hop_length = 512`, and  $n_{\text{mels}} = 128$ , followed by `AmplitudeToDB` conversion to log scale.
5. **SpecAugment (training only):** Frequency Masking (max 5 bins) and Time Masking (max 2 frames) are applied following Park et al. [10] to further improve generalization.

Data is partitioned into an 80/20 train/validation split using stratified sampling via `scikit-learn` [13] to ensure balanced class representation across both splits, with `stratify` flag. Labels are encoded from categorical strings to integers via a `LabelEncoder`, fitted on the training set labels to ensure a consistent mapping across splits.



## Chapter 3

# Architectures

### 3.1 Common Building Blocks

#### 3.1.1 Causal Convolutions and Residual Blocks

All convolutional components in this work are built on **causally constrained convolutions** [14]. A standard 1D convolution at time step  $t$  has access to both past and future inputs within its receptive field, which would constitute data leakage in a real-time streaming scenario and may distort the learned representations. Causal convolutions prevent this by applying dynamic left-padding proportional to the kernel size and dilation factor, ensuring that the kernel at position  $t$  sees only inputs at positions  $\leq t$ . Weight Normalization [15] is applied to each causal layer to stabilize gradient flow during training.

These layers are organized into **Residual Blocks**, following the design of He et al. [16]. Each block consists of two sequential causal convolution layers, each followed by ReLU activation and Dropout regularization. The input to the block is added back to its output via a skip connection to mitigate vanishing gradients; when input and output channel counts differ, a  $1 \times 1$  convolution is used to align dimensions before the addition. A final ReLU and Dropout are applied to the summed output.

### 3.1.2 Bidirectional Recurrent Blocks

Long Short-Term Memory networks [17] address the vanishing gradient problem of standard RNNs through gated memory cells that selectively retain or discard information across arbitrarily long sequences. Each LSTM cell maintains a hidden state  $h_t$  and a cell state  $c_t$ , updated at every time step through three learned gates: the *input gate*  $i_t$ , which controls how much new information is written to  $c_t$ ; the *forget gate*  $f_t$ , which controls how much of the previous cell state is retained; and the *output gate*  $o_t$ , which controls how much of  $c_t$  is exposed as the hidden state  $h_t$ .

In this work, LSTMs are used in their **bidirectional** variant [18]: two independent LSTM layers process the input sequence in opposite temporal directions, and their hidden states are concatenated at each time step. This allows the model to incorporate both past and future context when forming a representation at position  $t$ , effectively doubling the output dimensionality to  $2 \times d_h$ , where  $d_h$  is the hidden size of each directional layer. Layer Normalization [19] is applied after the recurrent stack to stabilize activations across the time dimension, mitigating the internal covariate shift that can otherwise slow convergence in deep recurrent architectures.

## 3.2 Temporal Convolutional Network (TCN)

The Temporal Convolutional Network [14] replaces recurrence entirely with stacked dilated causal convolutions, avoiding the sequential computation bottleneck of recurrent architectures. In this work, four Residual Blocks are stacked sequentially with the following fixed hyper parameters:

- Filter channels per layer: 128
- Kernel size: 3
- Dropout rate: 20%
- Activation: ReLU throughout

Following the residual stack, an **Adaptive Average Pooling** layer collapses the temporal dimension to a fixed-size vector, which is then passed to a **Fully Connected** classification head to produce the final per-class logits. Total trainable parameters:  $\approx 399\text{k}$ .

## 3.3 Long Short-Term Memory Network

Long Short-Term Memory networks [17] address the vanishing gradient problem of standard RNNs through gated memory cells that selectively retain or discard information across arbitrarily long sequences. The architecture used here processes the Mel Spectrogram as a temporal sequence of 128-dimensional frequency vectors through:

- Two stacked Bidirectional LSTM layers with hidden dimensionality 128 (256 effective due to bidirectionality).
- Inter-layer Dropout: 30%.

- Layer Normalisation [19] with `norm_shape = 256` applied after the recurrent stack.
- **Global Max Pooling** across the time dimension to extract the most salient hidden state.
- A two-layer Fully Connected classifier with ReLU and Dropout between layers.

Total trainable parameters:  $\approx 697k$ , the largest among the three architectures.

### 3.4 Convolutional Recurrent Neural Network (CRNN)

The CRNN combines convolutional and recurrent processing in sequence, following the hybrid structure proposed by Choi et al. [20] for audio classification tasks where both local spectral features and temporal context are relevant.

- **Feature Extractor:** Three Residual Blocks (128 channels, kernel size 3, 20% Dropout, ReLU) extract local acoustic patterns from the Mel Spectrogram, producing a temporally compressed feature map.
- **Sequence Modeller:** A single Bidirectional LSTM layer (hidden size 64, 20% Dropout) captures temporal context across the features produced by the CNN stage.
- **Aggregation:** Max Pooling across the time dimension selects the most informative features.



- **Classifier:** Two Fully Connected layers produce the final class scores.

Total trainable parameters:  $\approx 406\text{k}$ .



## Chapter 4

# Experiments, Results and Conclusion

### 4.1 Experimental Setup

All models were optimised using the Adam optimiser [21] with a `ReduceLROnPlateau` learning rate scheduler that halves the rate upon validation loss stagnation. Early stopping was applied to prevent overfitting. Table 4.1 summarises the training hyperparameters for each architecture.

TABLE 4.1: Training hyperparameters per architecture.

| Hyperparameter          | TCN   | LSTM   | CRNN   |
|-------------------------|-------|--------|--------|
| Batch Size              | 32    | 32     | 32     |
| Learning Rate           | 0.001 | 0.0001 | 0.0001 |
| Max Epochs              | 50    | 100    | 100    |
| LR Scheduler Factor     | 0.5   | 0.5    | 0.5    |
| Early Stopping Patience | 10    | 20     | 20     |

## 4.2 Results

Table 4.2 presents the test set performance of all three architectures.

TABLE 4.2: Test set performance comparison.

| Metric     | TCN          | LSTM  | CRNN  |
|------------|--------------|-------|-------|
| Accuracy   | <b>89%</b>   | 80%   | 83%   |
| Macro F1   | <b>0.887</b> | 0.748 | 0.742 |
| Parameters | <b>399k</b>  | 697k  | 406k  |

The TCN achieves the best performance across all metrics while remaining the most parameter-efficient model. The CRNN surpasses the LSTM in accuracy despite having roughly 40% fewer parameters. A **CoatNet** [22] model was also explored but did not achieve competitive performance, falling below the LSTM baseline, and was therefore excluded from the main comparison.

### 4.3 Discussion

The superiority of the TCN on this task aligns with the theoretical arguments of Bai et al. [14], who demonstrated that TCNs outperform recurrent models on many sequence modelling benchmarks, particularly when tasks do not require very long-range dependencies. Individual keystrokes are **highly transient events**: their discriminative information is concentrated in the rapid amplitude and spectral changes of the first few tens of milliseconds. Causal dilated convolutions efficiently cover a wide receptive field over this brief window without the sequential computation bottleneck of recurrence.

The LSTM, despite its greater parameter count, faces a complexity ceiling on this small dataset. Deeper recurrent stacks are prone to vanishing gradients even with gating mechanisms [17], and the temporal dependencies present in isolated 0.4-second keystroke clips are not long enough to justify multi-layer recurrence, resulting in a model that is both larger and less accurate than the TCN.

The CRNN represents a sensible architectural compromise: its convolutional front-end extracts per-key acoustic fingerprints reliably, while the LSTM back-end adds temporal context. For the *isolated* classification task studied here, however, that temporal context provides limited added value over a purely convolutional approach. The CRNN’s advantage would be more pronounced in a *continuous* typing scenario, where the LSTM could model the sequence of characters rather than a single keystroke in isolation [20].

A consistent finding across all three models is that **lean architectures generalise better under data-scarce conditions** [11]: larger, more expressive models suffer from data starvation and tend to overfit, whereas the architectures explored here achieve strong performance

without over-parameterisation.

## 4.4 Conclusion and Future Work

This project demonstrates that deep learning-based acoustic keystroke classification is both feasible and effective, achieving 89% test accuracy with a compact TCN model under constrained data conditions. These results confirm and extend prior findings in the ASCA literature [2], [3]: acoustic keylogging is a credible and practically accessible privacy threat that requires no system access, no specialised hardware, and no large-scale data collection.

Several directions could strengthen and extend this work:

- **Dataset Expansion:** Collecting data across diverse keyboards, microphones, and acoustic environments would improve robustness, addressing the environmental sensitivity identified as a primary limitation in the ASCA literature [1].
- **Continuous Sequence Modelling:** Extending the system to process continuous typing streams, rather than pre-segmented keystrokes, would bring it closer to a deployable attack scenario. The CRNN is well-positioned for this extension [20].
- **NLP Integration:** Pairing the acoustic classifier with an  $n$ -gram language model or a pre-trained transformer could resolve ambiguous characters, substantially improving end-to-end word reconstruction accuracy. Open-source tools such as Keytap [23] demonstrate this approach using classical signal processing; combining it with deep acoustic models is a natural next step.

- **Two-Stage Denoising Pipeline:** A frontend noise reduction model prior to classification would enable robust operation in realistic noisy environments without degrading the classifier’s core performance [11].





## Appendix A

# Code Listings

The source code for this project is available at:

<https://github.com/MVenticinque/KeyboardAudioDetection.git>

Key modules include:

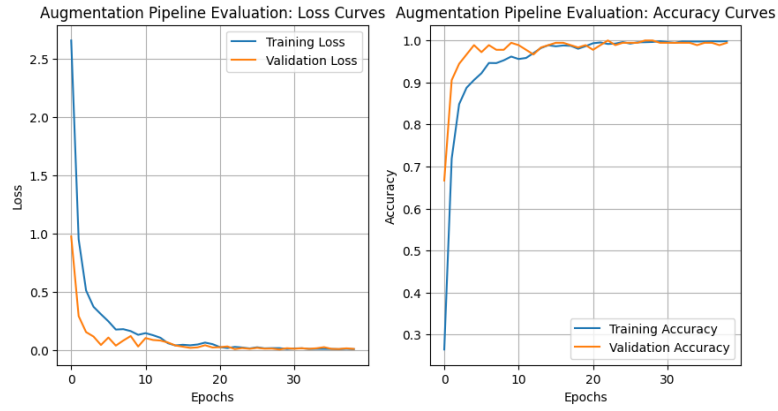
- `utils/peak_detection.py` — CWT-based transient detection and audio splitting (Algorithms 1 and 2).
- `utils/data_aug.py` — Signal-level data augmentation pipeline (Algorithm 3).
- `utils/dataset.py` — `KeyAudioDataset` class with Mel spectrogram extraction and `SpecAugment`.
- `utils/tcn.py`, `utils/lstm.py`, `utils/crnn.py` — Model definitions.
- `weights/tcn_best.pth`, `weights/lstm_best.pth`, `weights/crnn_best.pth` — Model weights.

- `slides.qmd`, `style.css` — Slides used to present the project. To be executed with `quarto render`.

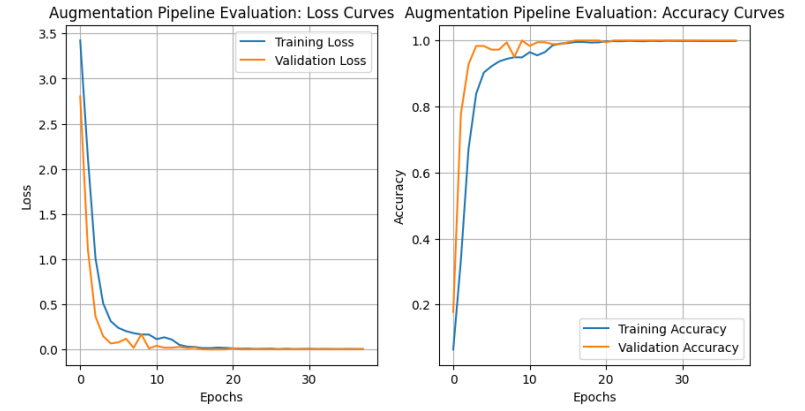
## Appendix B

### Plots

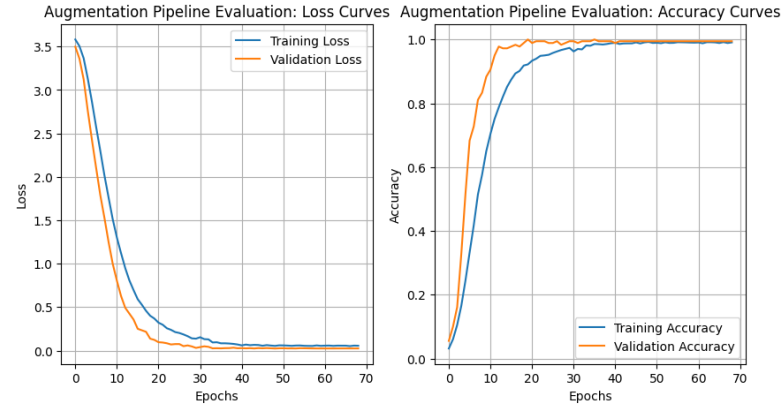




(A) TCN Loss and Accuracy

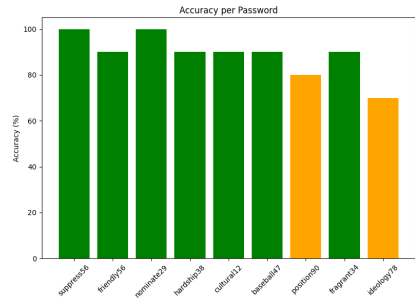


(B) LSTM Loss and Accuracy

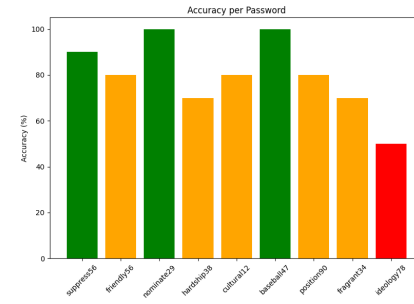
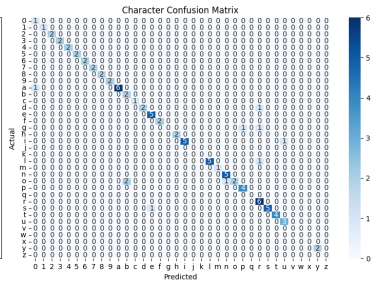


(C) CRNN Loss and Accuracy

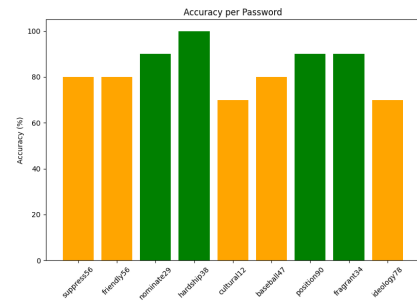
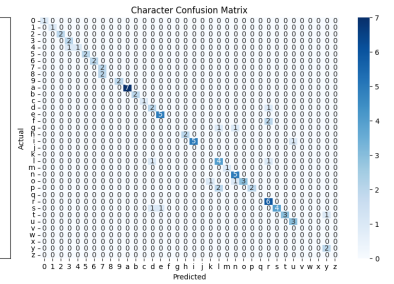
FIGURE B.1: Cross Architecture comparison of training performances



(A) TCN test Accuracy



(B) LSTM test Accuracy



(C) CRNN test Accuracy

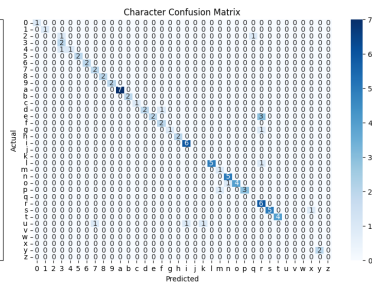


FIGURE B.2: Cross Architecture comparison of testing performances

# Bibliography

- [1] D. Asonov and R. Agrawal, “Keyboard acoustic emanations,” in *IEEE Symposium on Security and Privacy*, IEEE, 2004, pp. 3–11.
- [2] L. Zhuang, F. Zhou, and J. D. Tygar, “Keyboard acoustic emanations revisited,” *ACM Transactions on Information and System Security*, vol. 13, no. 1, pp. 1–26, 2009.
- [3] J. Harrison, E. Toreini, and M. Mehrnezhad, “A practical deep learning-based acoustic side channel attack on keyboards,” in *IEEE European Symposium on Security and Privacy Workshops*, IEEE, 2023.
- [4] M. O. Spata, V. M. Russo, A. Ortis, and S. Battiato, “A new pipeline for snooping keystroke based on deep learning algorithm,” *IEEE Access*, vol. 13, pp. 24 498–24 514, 2025.
- [5] L. Crestel and P. Esling, “Live orchestral piano, a system for real-time orchestral music generation,” in *Sound and Music Computing Conference (SMC)*, 2018.
- [6] B. McFee et al., “Librosa: Audio and music signal analysis in Python,” 2015.

- [7] M. O. Spata, *Kad*, <https://github.com/spatam/kad>, 2025.
- [8] G. Lee, R. Gommers, F. Waselewski, K. Wohlfahrt, and A. O’Leary, “PyWavelets: A Python package for wavelet analysis,” *Journal of Open Source Software*, vol. 4, no. 36, p. 1237, 2019.
- [9] J. Robert, *Pydub: Manipulate audio with a simple and easy API*, <https://github.com/jiaaro/pydub>, 2011.
- [10] D. S. Park et al., “SpecAugment: A simple data augmentation method for automatic speech recognition,” in *Interspeech*, 2019.
- [11] J. Salamon and J. P. Bello, “Deep convolutional neural networks and data augmentation for environmental sound classification,” *IEEE Signal Processing Letters*, vol. 24, no. 3, pp. 279–283, 2017.
- [12] S. Hershey et al., “CNN architectures for large-scale audio classification,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017.
- [13] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [14] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” in *arXiv preprint arXiv:1803.01271*, 2018.
- [15] T. Salimans and D. P. Kingma, “Weight normalization: A simple reparameterization to accelerate training of deep neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016, pp. 901–909.
- [16] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.



- [17] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [18] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [19] J. L. Ba, J. R. Kiros, and G. E. Hinton, *Layer normalization*, arXiv preprint arXiv:1607.06450, 2016.
- [20] K. Choi, G. Fazekas, M. Sandler, and K. Cho, “Convolutional recurrent neural networks for music classification,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, 2017.
- [21] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [22] Z. Dai, H. Liu, Q. V. Le, and M. Tan, “CoAtNet: Marrying convolution and attention for all data sizes,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [23] G. Gerganov, *Keytap: Keyboard acoustic eavesdropping*, <https://github.com/ggerganov/kbd-audio>, 2019.